SkillPass: Technical Report

Final Version of the Product (In Progress) · A trainer-verified digital skills passport for informal apprentices in Nigeria

Fejiro Obiku · BSc Software Engineering, African Leadership University · Supervisor: Tunde Isiaq Gbadamosi · October 2026

1. Scope alignment and feature validation

SkillPass is a full-stack web application: a React and TypeScript single-page app talks to a REST API (Node.js 22, Express 5) backed by PostgreSQL. Table 1 maps every module in the approved proposal (Table 6 of the proposal) to its current state. Screenshots are in docs/screenshots and source paths are relative to the repository root.

Planned feature	Intended purpose	Current status	Evidence
Accounts and roles	Role-based access; trainers approved before issuing; apprentice linked to one trainer	Completed	backend/src/routes/auth.js, middleware/auth.js; 13_register.jpg; tests: "authentication", "bootstrap: what each role is allowed to see"
Skill framework	Checklists per trade by level; trainers propose skills; advanced skills need a co-signer	Completed	routes/skills.js, routes/admin.js; Trades & Skills page; tests: "skills and trades"
Credential issuing	Trainer records an observed skill with evidence; signed, timestamped, unique ID	Completed	routes/credentials.js, routes/evidence.js, lib/signing.js; 03_issue_credential.jpg; tests: "issuing a credential"
Skills passport	Shareable, printable record of verified skills	Completed	routes/passport.js (revocable link + QR); 05_apprentice_passport.jpg; tests: "public passport"
Public verification	Anyone checks a credential by ID, link or QR, no login	Completed	routes/verify.js; 01_verify_valid.jpg, 02_verify_revoked.jpg; tests: "public verification"
Integrity safeguards	Evidence first, co-signing, issuing limit, random audits, flag/revoke, trust score, append-only log	Completed	lib/scoring.js, lib/integrity.js, lib/auditChain.js; 09_review_queue.jpg, 10_audit_log.jpg; tests: "flags and disputes", "risk and misconduct", "daily issuing limit"
Employer feedback	Ratings linked to credentials feed the trainer trust score	Completed	routes/feedback.js, routes/verify.js (trust rating); tests: "employer ratings"
Job referral	Rank verified apprentices by skill and distance; notify by SMS or in-app alert	Partial	lib/matching.js, routes/jobs.js; 07_job_referrals.jpg, 08_employer_jobs.jpg. In-app alerts work; SMS runs in console mode until a Termii sender ID is approved
Administration	Approve trainers, manage skills, review flags and audit results	Completed	routes/admin.js; 09_review_queue.jpg, 11_pilot_metrics.jpg; tests: "admin extras"
Pilot deployment	Run the pilot with real users in Lagos	Not implemented	Waiting for ethics approval (submitted 23 Oct 2026); deployment plan in README

Table 1. Planned features from the approved proposal and their current status.

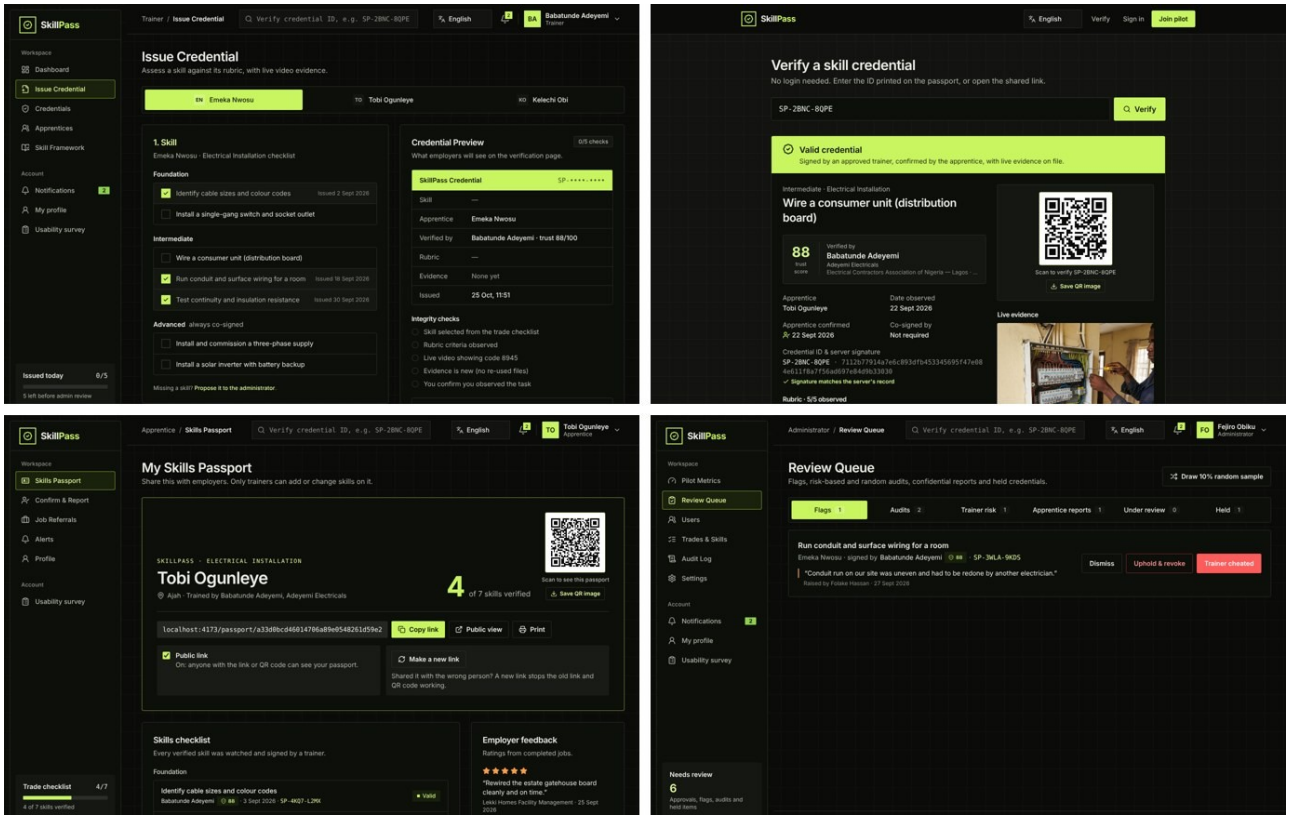


Figure 1. Evidence of core features: trainer issues a credential with a live-video code (top left); public verification with signature check and QR code (top right); apprentice passport (bottom left); administrator review queue (bottom right).

Justified scope changes

Added after supervisor and ethics feedback. Phone verification by SMS code with email verification and password reset (accounts must be real people); Nigerian Pidgin alongside English; an offline upload queue for weak signal; apprentice-controlled passport links; a confidential report channel for apprentices; consent withdrawal that removes personal data (Nigeria Data Protection Act 2023); a hash-chained audit log; and expiring signed links for evidence files, so the privacy claims in the ethics application are enforced by the code.

Partial or deferred. Real SMS delivery is implemented (TermiiSmsChannel) but switched to console mode, because Nigerian networks require an approved sender ID for messages to reach Do-Not-Disturb numbers. Deployment and the live pilot wait for ethics approval. Payments, NIN integration, native apps, blockchain anchoring and NSQ integration stay out of scope, as stated in the proposal.

2. Technical functionality

Architecture. The frontend (src/) is organised by role (pages/trainer, apprentice, employer, admin, public) with shared components, an auth context and a typed API client. The backend (backend/src) separates HTTP routes (one file per area) from domain logic in lib/ (scoring, matching, signing, storage, messaging, audit chain, privacy), with middleware for authentication and errors. Every request body is validated with Zod; every protected route checks the JWT and the role on the server; database writes that must stay consistent run in transactions.

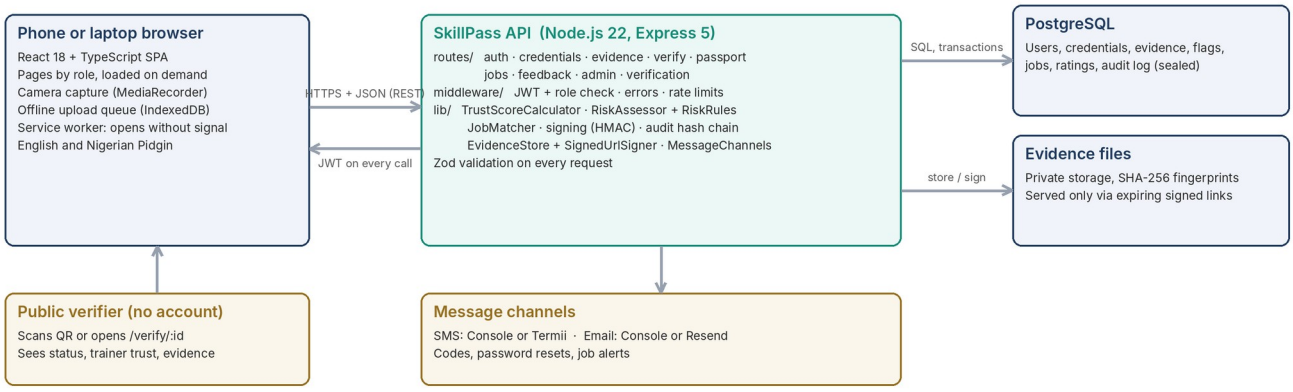


Figure 2. System architecture: the browser app talks to the REST API over HTTPS; domain logic lives in classes under backend/src/lib.

Core flow: issuing and verifying a credential. (1) The trainer asks the API for a one-time challenge code, valid for 15 minutes. (2) The app records a live video showing the code; the upload is checked by magic bytes, not the browser's claimed type, hashed with SHA-256 on the server, and refused if that hash already backs another credential. (3) The trainer ticks the rubric criteria and signs; the API locks the trainer row (SELECT … FOR UPDATE) so two requests cannot both slip under the daily limit, applies probation and co-sign rules, signs the credential with HMAC-SHA256 and writes an audit entry. (4) The apprentice confirms, and only then is the credential valid. (5) An employer opens /verify/:id, where the API recomputes the signature and the trainer's live trust score; the time taken and the employer's trust rating feed the pilot metrics.

Testing with different inputs. The API returns specific, actionable errors and the UI shows them: wrong password or suspended account (401/403); an under-18 date of birth at sign-up (400, "SkillPass pilot participants must be 18 or older"); a reused challenge code (400); a duplicate evidence file (409, also written to the audit log); a credential beyond the daily limit (held for review); confirming someone else's credential (404); an unknown credential ID ("Not found" page); an edited or expired evidence link (403); a tampered credential signature ("Signature does not match"). All of these are covered by automated tests.

3. Testing strategies and results

Strategy	Why chosen	How conducted	Result and improvement made
Unit testing	The integrity logic (trust, risk, matching, signed links) must be exactly right and is easy to test in isolation	29 Node test-runner tests on the domain classes with fixed clocks and fake inputs; no database	29/29 pass. Found a time-boundary edge in the signed-link cache test; fixed the test clock
Integration (API + DB)	Integrity rules depend on SQL, transactions and locks working together	81 tests run the Express app against a real PostgreSQL database, wiped and re-seeded per run	81/81 pass. Found a demo seed notification that told a trainer about a pending audit (it would let them prepare); replaced it
End-to-end UI	Shows that real screens and the real API work together	21 Vitest + Testing Library tests render the app against the running API	21/21 pass. A full browser walkthrough of 47 page/role combinations found a passport crash (variable shadowing) and phone-width overflow; both fixed
Security	Evidence and records are sensitive; claims in the ethics application must hold	Forged, edited and expired evidence links; tampered signatures; audit-log tampering with triggers disabled; access across roles	All pass. Found that evidence was served by a permanent address; added expiring HMAC-signed links
Performance and compatibility	Users have low-cost Android phones on 3G/4G	Playwright + Chrome DevTools throttling (CPU and network); API load test; 4 screen sizes (Section 4)	Cut JavaScript by 44% and first-load time by 16–27% on a low-end phone (Section 4)

Table 2. Testing strategies. Totals: 131 automated tests (110 backend, 21 frontend), all passing; TypeScript and ESLint report 0 errors.

4. Performance and compatibility

Method. The production build was served with gzip, as Netlify or Vercel would, and each page was loaded with an empty cache 5 times per profile in Chromium, using Chrome DevTools Protocol throttling. Medians are reported. "Ready" is the time until the page's main content is visible.

Profile and page	Before: ready	After: ready	JS before → after	Change in ready time
Low-end Android (6× CPU, 3G): sign-in	2.69 s	1.96 s	243 → 135 KB	27% faster
Low-end Android: employer verifies a credential	3.23 s	2.54 s	243 → 135 KB	21% faster
Low-end Android: apprentice passport (signed in)	4.18 s	3.50 s	243 → 146 KB	16% faster
Mid-range Android (4× CPU, 4G): sign-in	1.41 s	0.91 s	243 → 135 KB	35% faster
Mid-range Android: apprentice passport	1.50 s	1.34 s	243 → 146 KB	11% faster
Desktop broadband: sign-in	0.44 s	0.43 s	243 → 135 KB	No real change

Table 3. Page performance before and after on-demand page loading (median of 5 cold loads). The gain is largest on slow phones and networks, where it matters for this pilot; on fast desktops the difference is negligible. JavaScript heap on the low-end verify page fell from 5.6 to 4.3 MB.

What changed and why. The first build shipped one 574 kB JavaScript bundle. Splitting every page into its own file reduced the download but, measured on 3G, made content appear later: the browser fetched the main file, then checked the account, and only then requested the page file. The final version keeps the public entry pages (sign-in, register, verify, passport) in the main bundle and, for a signed-in user, starts fetching their role's pages in parallel with the account check (src/pageLoaders.ts). The verify page, the target metric in the proposal (under 1 minute), is ready in 2.5 s on the slowest profile.

API under load. With 10 concurrent clients on a 2-core test machine: public verification took a median of 34 ms (p95 81 ms) at about 260 requests per second; an apprentice's full data load 47 ms (p95 59 ms); the heaviest request, an administrator's data with live trust and risk for every trainer, 147 ms (p95 192 ms). There were no failures.

Compatibility. Every screen was checked at 360 px (low-end Android), 390 px (iPhone-size), 768 px (tablet) and 1366 px (laptop) for horizontal overflow; after fixes, none overflows (docs/screenshots/compat_*.jpg). Automated runs used Chromium, the engine behind Chrome, Edge and Opera, the browsers most common on Android in Nigeria. Firefox and Safari were not available in the test environment and are shown working in the walkthrough video. The app targets modern evergreen browsers (ES2020, CSS custom properties) and needs no plug-ins; the camera uses the standard MediaRecorder API, and a browser that cannot record gets a clear message instead of a broken screen.

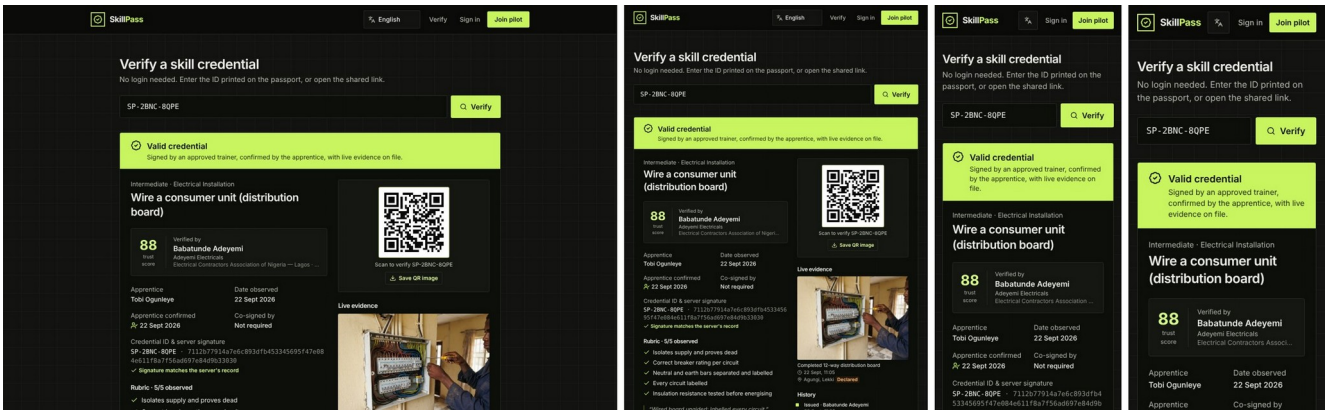


Figure 3. The public verification page at 1366, 768, 390 and 360 px wide: no horizontal scrolling, header kept on one line.

5. Code quality, OOP and algorithms

Object-oriented design

Class or module	OOP principle shown	Benefit
MessageChannel → HttpChannel → TermiiSmsChannel, ResendEmailChannel; ConsoleChannel (lib/messaging.js)	Abstraction (abstract base refuses direct use), inheritance, polymorphism: callers only call send()	Switch SMS or email provider in .env; adding Africa's Talking is one subclass. Tested with a fake fetch
RiskRule → FastIssuingRule, PassesEveryoneRule, ClusteredDayRule, PoorRatingsRule, OpenReportsRule; RiskAssessor (lib/scoring.js)	Strategy pattern, open/closed principle: new warning signs are added as classes without editing the assessor	Rules are tested one by one; a test injects a custom NightShiftRule to prove polymorphism
TrustScoreCalculator (lib/scoring.js)	Encapsulation: weights validated and frozen; constants named	Weights can be tuned during the pilot without touching the formula
JobMatcher (lib/matching.js)	Encapsulation of matching weights and the distance cap	Configurable per trade or city; validated inputs
EvidenceStore → LocalDiskStore; SignedUrlSigner (lib/storage.js, lib/signedUrls.js)	Abstraction with dependency injection; private methods (#mac) hide the key	A cloud store (Supabase) can replace disk storage with one subclass; links can't be forged

Class or module	OOP principle shown	Benefit
React components and contexts (src/components, src/contexts)	Composition, separation of concerns, reusable components (PassportView, ScoreRing, EvidenceCapture)	The same passport view serves the apprentice, trainer, employer and public pages

Table 4. Object-oriented structure. The existing function names (computeTrust, rankMatches, sendSms…) remain as thin wrappers, so no route had to change.

Algorithms and technical problem-solving

- **Job matching:** haversine great-circle distance plus a weighted score (70% skills held, 30% closeness within 30 km), using Set look-ups, O(n·k + n log n) for n apprentices and k skills.
- **Fraud signals:** one pass over time-sorted credentials finds pairs signed minutes apart, O(n log n); a hash map counts credentials per Lagos calendar day, O(n).
- **Unbiased random audits:** a Fisher–Yates shuffle driven by a cryptographic random source (crypto.randomInt) selects the audit sample, so every credential is equally likely to be picked and the choice cannot be predicted.
- **Tamper evidence:** each audit entry stores HMAC(previous hash, entry), forming a hash chain; verification walks the chain in O(n) and pinpoints the first broken link. Credentials and evidence links are HMAC-SHA256 signed and compared in constant time (timingSafeEqual) to resist timing attacks.
- **Data integrity under concurrency:** row-level locks and transactions enforce the daily issuing limit and probation rules even when two requests arrive at the same moment; a unique index on the evidence hash blocks reuse in O(1).
- **Weak-signal resilience:** an IndexedDB queue keeps failed uploads per user and retries them when the connection returns; each evidence link expiry is rounded to the minute so repeated loads hit the browser cache.

Quality practices

TypeScript strict checks and ESLint (0 errors); consistent naming (classes in PascalCase, functions and variables in camelCase, files by feature); folders by responsibility rather than file type; input validation at every API boundary; configuration validated at start-up, which refuses to run with missing or reused secrets; comments explain why, not what; over 20 commits with descriptive messages; and a README with installation steps, versions and repository structure. Known limitations and next steps: 44 lint warnings (mostly unused variables in older UI code), real SMS delivery awaiting an approved sender ID, Firefox and Safari performance not yet measured automatically, and the live pilot pending ethics approval.